



Factory Method

Padrão de Projeto Criacional

Maxuel • Alex • Caio • João Henrique • Vinícius Adriel

INSTITUTO FEDERAL Alagoas

Introdução

O que é?

O Factory Method é um padrão de projeto criacional que fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo de objetos que serão criados.

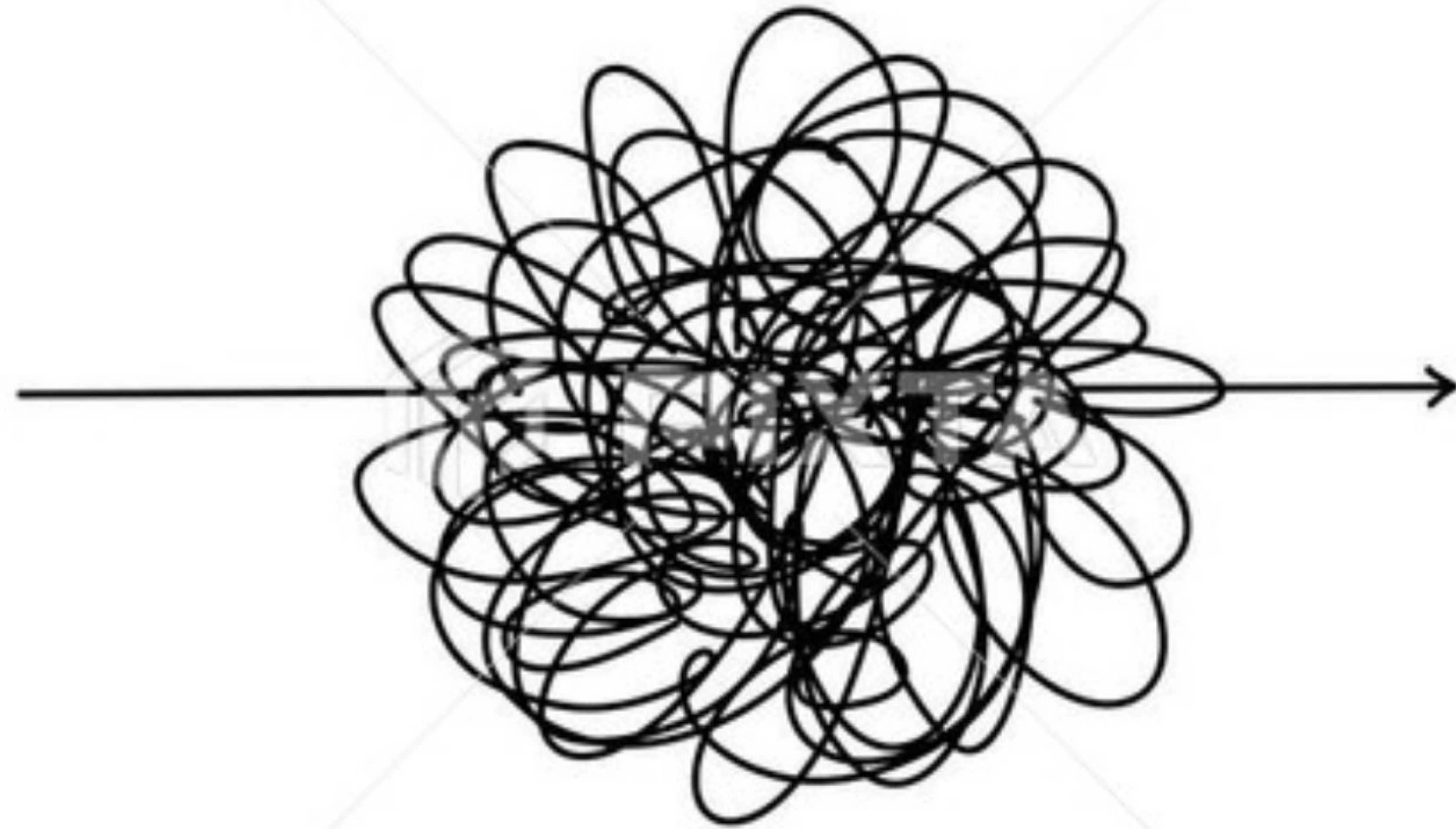
Objetivo Principal

Delegar a lógica de instanciação para as subclasses. Isso promove o desacoplamento, pois o código cliente não precisa saber exatamente qual classe concreta está sendo instanciada.

O Problema

Sem o padrão, o código tende a ficar **altamente acoplado** e difícil de manter.

- O sistema fica preso a uma única classe concreta (ex: Caminhão).
- Surgem inúmeros blocos if/else para decidir qual objeto criar.
- Para adicionar um novo tipo, é necessário alterar o código existente, violando o princípio Aberto/Fechado (Open/Closed).

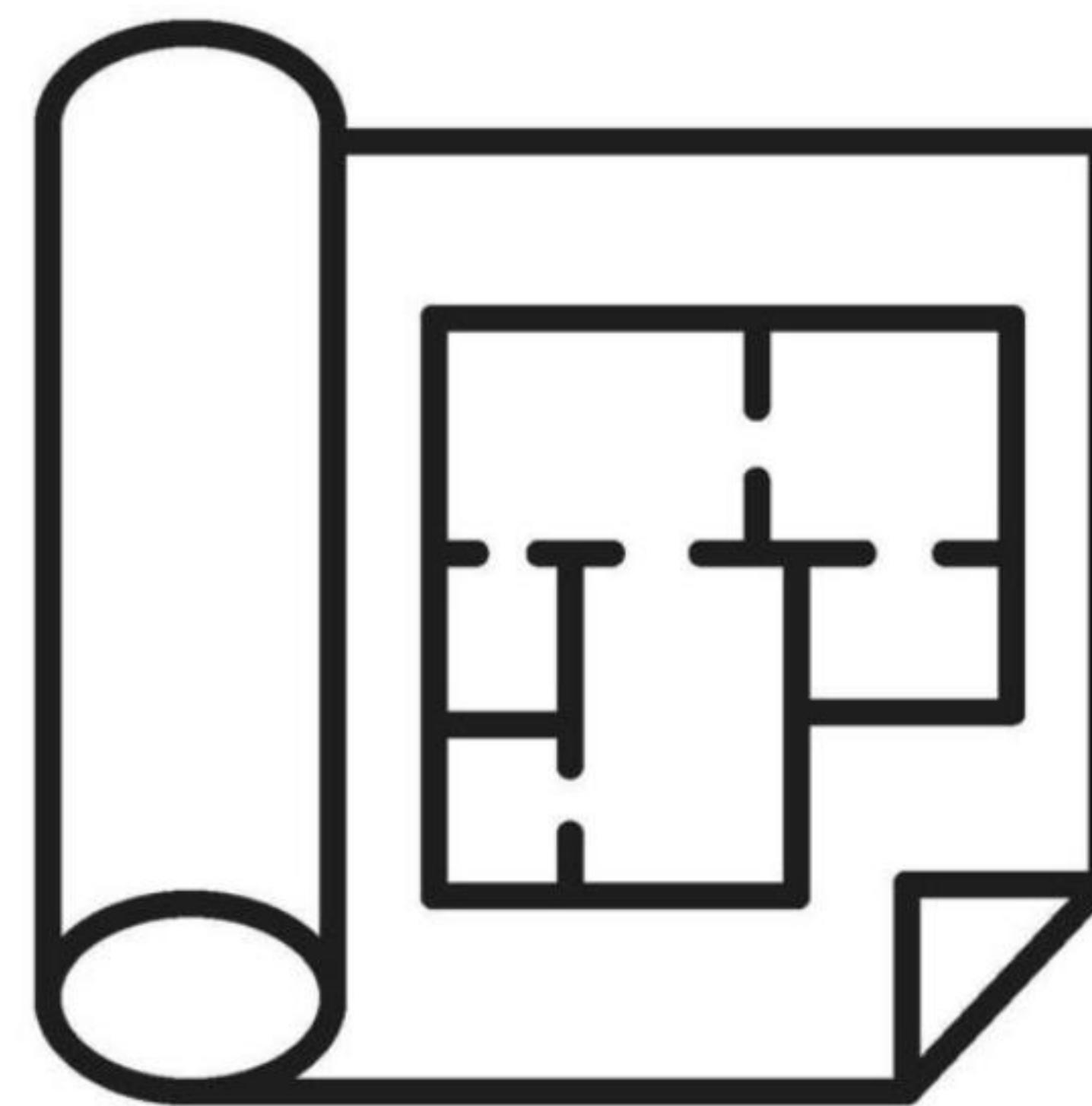


A Solução

O padrão sugere substituir a criação direta de objetos (usando o operador `new`) por chamadas a um **método fábrica** especial.

"As subclasses podem sobrescrever esse método fábrica para retornar diferentes tipos de produtos."

Todos os produtos devem seguir uma interface comum, garantindo que o código cliente funcione com qualquer produto retornado pela fábrica.



Estrutura do Padrão



Produto

Declara a interface comum para todos os objetos que podem ser criados.



Produto Concreto

Implementações distintas da interface do produto (ex: Navio, Caminhão).



Criador

Declara o método fábrica que retorna novos objetos do produto.



Criador Concreto

Sobrescreve o método fábrica para retornar um tipo específico de produto.

Exemplo: Logística



LOREM IPSUM

Transporte Marítimo vs. Rodoviário

Imagine um aplicativo de logística. Inicialmente, ele lida apenas com transporte por caminhões. Depois, precisa suportar navios.

- **Interface:** Transporte (método entregar())
- **Fábrica Rodoviária:** Cria objetos Caminhão.
- **Fábrica Marítima:** Cria objetos Navio.

O cliente apenas chama `fábrica.criarTransporte()` sem se preocupar com a classe exata.

Prós e Contras

Vantagens

- Evita acoplamento forte entre o criador e os produtos concretos.
- **Single Responsibility Principle:** O código de criação fica em um local único.
- **Open/Closed Principle:** Novos tipos de produtos podem ser introduzidos sem quebrar o código existente.

Desvantagens

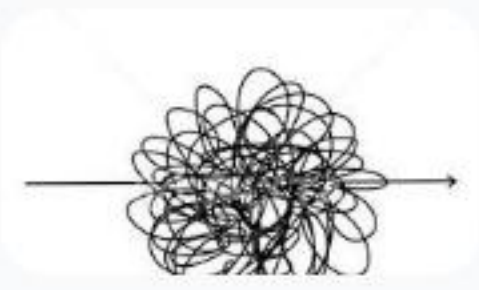
- O código pode se tornar mais complicado, pois exige a criação de muitas novas subclasses para implementar o padrão.
- Pode ser um "exagero" (overkill) se o sistema for simples demais.

Obrigado!

Perguntas?

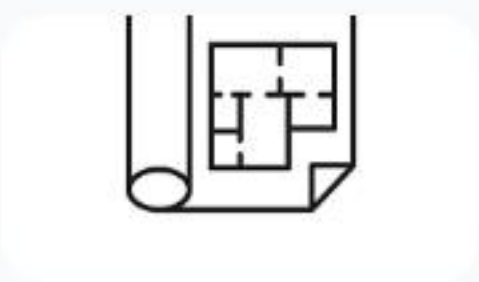
 INSTITUTO FEDERAL Alagoas

Image Sources



<https://en.pimg.jp/111/825/302/1/111825302.jpg>

Source: www.pixtastock.com



https://static.vecteezy.com/system/resources/previews/055/733/497/non_2x/this-minimalist-icon-presents-a-blueprint-that-signifies-the-critical-aspects-of-construction-and-renovation-effectively-embodying-the-essence-of-design-and-planning-within-the-construction-industry-vector.jpg

Source: www.vecteezy.com



https://img.freepik.com/premium-vector/logistic-delivery-concept-cargo-truck-plane-ship-truck-container-vector-illustration_122759-975.jpg

Source: www.freepik.com